附录一：CAN 通信协议——CiH408

1. 关节通信介绍

具身智能机器人由关节组成，每个关节及关节至网关间通过 CAN 通信进行关节设置、查询、指令控制及状态监测，CAN 通信协议以使人形机器人实时控制全身关节为目的。

CiH408 协议参考了 CiA301 及 CiA402 协议，鉴于 CIA402 在使用中的不足进行更加适配人形机器人的完善。其不足之处在于：1.CiA402 不具备人形机器人常用的力位混合模式及对应参数，需要自行设置；2.如果使用 CiA402 进行运动控制中的力位混合模式时，由于每个变量长度有规定值，现有网关条件下（自研网关）每次控制需要发送两包数据。主要是由于自研网关需要 1 发送对应 1 接收，无法使用 pdo 使每包报文包含多种信息。

人形机器人的控制信号的特殊性要求：需要具有实时性，使通信频率大于 400Hz，控制运动时所有控制参数同报文发送，需要具有实时返回以检测机器人状态。自定义协议优势：可在单通道对整臂/整腿关节控制；具有帧头判断以避免错误控制；具有控制自增判断以检测是否丢包；具有测试帧功能；具备 bootloader 升级功能；相比竞品关节使用 CAN2.0，使用 CANFD 数据域波特率更高，实现提升控制频率与精度。

2. CAN 通信参数

CAN 类型：CAN FD（若接收 CAN2.0B 报文，无法控制电机转动，但可进行电机 ID 设置，接收 CAN2.0B 则返回 CAN2.0B）。

波特率：仲裁域 1Mbps；数据域 5Mbps。

CANID：标准标识符（0x01~0x7FF）。另有关节 ID，在查询 ID 及重置 ID 功能时发送 CANID=0x000；其余关节设置与电机控制时 CANID=关节 ID；Bootloader 升级时 CANID=关节 ID+0x400。下文中出现的"ID"，若无特殊说明，均指"关节 ID"。

数据长度（dlc/dlen）：上限 16 字节，建议按下文中规定的收发长度设定 CAN 长度。

通信模式问答模式：向关节发送报文，均会得到反馈，且仅有更改 ID 功能时，发送与接收的 CANID 不同。

数据域中，若多个字节数据拼成 8 位以上数值，则按大端模式，即若目标速度为 uint32 且由 Byte1-4 拼成，按 Byte1 数据左移 24 位、Byte2 数据左移 16 位、Byte3 数据左移 8 位、Byte4 数据相互或运算得到 uint32 目标速度。下文表格中按 CAN 报文 Byte1~16 表示。

帧格式均为为标准帧。

3. 关节设置指令

本章中指令为对关节设置，不涉及使关节电机运动控制，适用于工厂出厂检测、 关节编码器标定、测试部门测试、装配部门装配机器人及运动控制时使用。

3.1 关节 ID 查询

查询 CAN 线上关节 ID（发送） CANID: 0x000　dlc: 8			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x00	0x00	0x00	0x00
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
0x00	0x00	0x00	0x00

查询 CAN 线上关节 ID（接收） CANID: 0x000　dlc: 2			
Byte 1	***Byte 2***		
0x80	关节 ID		

建议 CAN 总线只连单关节，如果多关节级联时发送查询 ID 指令，会让所有关节返回 ID，进而可能导致上位机收到大量报文从而使其无法分辨对应关节的 ID，因此建议只对单个关机连线时发送查询 ID 指令。

该指令与下文关节 ID 重置指令均需要向关节发送 8 字节，以防止出现 CAN 线干扰导致关节 ID 误重置。

3.2 关节 ID 重置

对 CAN 线上关节 ID 重置为 1（发送） CANID: 0x000　dlc: 8			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x01	0xD5	0xA3	0xF1
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
0xE8	0xC2	0xB7	0x34

对 CAN 线上关节 ID 重置为 1（接收） CANID: 0x000　dlc: 2			
Byte 1	***Byte 2***		
0x81	0x01		

如果多关节级联时发送重置 ID 指令，会让所有关节 ID 都变成 1，控制电机时有且仅有 ID1 能被控制，发送 1 包控制指令后会返回关节数的状态返回数据，由于网关性质，无法发送 1 包数据接收超过 1 包数据，因此不建议对多关节发送关节重置指令。

3.3 关节 ID 修改

对 CAN 线上关节 ID 改变（发送） CANID: 关节旧 ID　dlc: 3			
Byte 1	***Byte 2***	***Byte 3***	
0x02	旧 ID	新 ID	

对 CAN 线上关节 ID 改变（接收） CANID: 关节新 ID　dlc: 3			
Byte 1	***Byte 2***	***Byte 3***	
0x82	旧 ID	新 ID	

对 CAN 线上一个 ID 改为另一个 ID 号，注意不要改成与其他关节相同 ID。发送的报文

CANID 与第二字节均为旧 ID 以防止出现错误。

3.4 关节零点设置

对关节进行零点设置（发送） CANID: 关节 ID　dlc: 1			
Byte 1			
0x03			
对关节进行零点设置（接收） **CANID: 关节 ID　dlc: 6**			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x83	成功/失败	零点位置 (float)31~24 位	零点位置 (float)23~16 位
Byte 5	***Byte 6***		
零点位置 (float)15~8 位	零点位置 (float)7~0 位		

该指令给关节发送后，关节会将当前位置值设置为零点并存储至 Flash 中，重新上电依然使用更新后的零点。设置成功第二个字节返回 0x01，失败返回 0x00。

3.5 风扇控制

风扇开关（发送） CANID: 关节 ID　dlc: 2			
Byte 1	***Byte 2***		
0x04	启动/停止		

启动为 0x01，停止为 0x00，关节上电后默认停止状态。仅有部分关节具备风扇接口，具体见各型号关节说明书。

风扇开关（接收） CANID: 关节 ID dlc: 2			
Byte 1	***Byte 2***		
0x84	启动/停止		

3.6 MCUID 查询

MCUID 查询（发送） CANID: 关节 ID dlc: 1			
Byte 1			
0x05			

MCUID 查询（接收） CANID: 关节 ID dlc: 13			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x85	MCUID 95~88 位	MCUID 87~80 位	MCUID 79~72 位
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
MCUID 71~64 位	MCUID 63~56 位	MCUID 55~48 位	MCUID 47~40 位
Byte 9	***Byte 10***	***Byte 11***	***Byte 12***
MCUID 39~32 位	MCUID 31~24 位	MCUID 23~16 位	MCUID 15~8 位
Byte 13			
MCUID 7~0 位			

每个 MCU 均有其独特 ID 存在 Flash 中，无法擦除或修改，共有 96 位，以 12 个 uint8 表示，允许 CAN 通信查询此独特 ID 以做记录用。

4. 关节控制指令

关节控制指令中，包含对关节运动的启动、停止、进入各种控制模式及其对应的输入参数，CANID 为目标关节 ID 号，收到的关节数据反馈 ID 也为目标关节 ID 号。如果一种数据的长度如果超过一个字节(8bits)，全部按照大端模式进行拼接。

关节驱动板中，有些控制数据为 float 类型，但由于 CAN 报文发送 uint 型数据，若发送某个数据用了 4 字节，则直接将其转化为 float 型，若发送数据小于 4 字节，则可能需要将对应的 uint/int 进行缩放处理变为有小数点的 float 型，具体缩放系数及示例下文会给出。

关节控制指令中，除启动/停止控制指令外，均返回下一章的状态返回报文，具体解析及切换返回报文见下一章。

关节控制指令中，最短间隔发送时间为 1ms，最长为 100ms，除发送停止控制功能外，如果关节在收到 CAN 报文 100ms 后未再收到 CAN 报文，则关节状态自动进入停止状态。

4.1 关节启动/停止控制

关节启动/停止（发送） CANID：关节 ID　dlc: 2			
Byte 1	***Byte 2***		
0x10	启动/停止		

关节启动/停止（接收） CANID：关节 ID　dlc: 2			
Byte 1	***Byte 2***		
0x90	启动/停止		

关节使能代码，进入各种关节控制模式前需要先将关节使能，启动为 0x01，停止为 0x00。

4.2 力位混合模式控制

力位混合模式（发送） CANID：关节 ID　dlc: 16			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x11	KP(uint16) 15~8 位	KP(uint16) 7~0 位	KD(uint16) 15~8 位
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
KD(uint16) 7~0 位	目标位置 (float)31~24 位	目标位置 (float)23~16 位	目标位置 (float)15~8 位
Byte 9	***Byte 10***	***Byte 11***	***Byte 12***
目标位置 (float)7~0 位	目标速度 (float)31~24 位	目标速度 (float)23~16 位	目标速度 (float)15~8 位
Byte 13	***Byte 14***	***Byte 15***	***Byte 16***
目标速度 (float)7~0 位	前馈力(int16) 15~8 位	前馈力(int16) 7~0 位	自增计数

在电机处于启动状态时，会根据输入参数进入力位混合模式转动，KP、KD 值为 uint16 型，范围从 0~6553.5，即 uint16 的数值为目标 KP/KD 乘 10 后的值（若发送 0x0064，转化为十进制为 100，即目标值为 10.0）；目标位置、目标速度为 float 型；前馈力为 int16 型，范围为-32768~32767；自增计数用来检测丢包，在测试帧 2 中会以发送值原样返回。

4.3 位置模式控制

位置模式（发送） CANID：关节 ID　dlc: 14			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x12	目标位置 (float)31~24 位	目标位置 (float)23~16 位	目标位置 (float)15~8 位
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***

目标位置 (float)7~0 位	速度阈值 (float)31~24 位	速度阈值 (float)23~16 位	速度阈值 (float)15~8 位
Byte 9	***Byte 10***	***Byte 11***	***Byte 12***
速度阈值 (float)7~0 位	电流阈值 (float)31~24 位	电流阈值 (float)23~16 位	电流阈值 (float)15~8 位
Byte 13	***Byte 14***		
电流阈值 (float)7~0 位	自增计数		

在电机处于启动状态时，会根据输入参数进入位置模式转动，目标位置、速度阈值、电流阈值为 float 型；自增计数用来检测丢包，在测试帧 2 中会以发送值原样返回。

4.4 速度模式控制

速度模式（发送） CANID：关节 ID　dlc: 10			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x13	目标速度 (float)31~24 位	目标速度 (float)23~16 位	目标速度 (float)15~8 位
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
目标速度 (float)7~0 位	电流阈值 (float)31~24 位	电流阈值 (float)23~16 位	电流阈值 (float)15~8 位
Byte 9	***Byte 10***		
电流阈值 (float)7~0 位	自增计数		

在电机处于启动状态时，会根据输入参数进入速度模式转动，目标速度、电流阈值为 float 型；自增计数用来检测丢包，在测试帧 2 中会以发送值原样返回。

4.5 电流控制模式

电流模式（发送） CANID：关节 ID　dlc: 6			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x14	目标电流 (float)31~24 位	目标电流 (float)23~16 位	目标电流 (float)15~8 位
Byte 5	***Byte 6***		
目标电流 (float)7~0 位	自增计数		

在电机处于启动状态时，会根据输入参数进入电流模式转动，目标电流为 float 型；自增计数用来检测丢包，在测试帧 2 中会以发送值原样返回。

4.6 VF 控制

VF 模式（发送） CANID：关节 ID　dlc: 9			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x15	输出电压 (float)31~24 位	输出电压 (float)23~16 位	输出电压 (float)15~8 位
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
输出电压 (float)7~0 位	输出频率 (float)31~24 位	输出频率 (float)23~16 位	输出频率 (float)15~8 位
Byte 9			
输出频率 (float)7~0 位			

在电机处于启动状态时，会根据输入参数进入 VF 模式，输出电压、输出电流为 float 型，需连续发送使电机连续转动，停止发送则电机停止转动。

4.7 查询当前信息

查询（发送） CANID: 关节 ID dlc: 1			
Byte 1			
0x17			

该报文仅用来查询关节数据，不会对关节改变现有运动状态，会根据返回报文类型返回状态报文（详见下一章）。

5. 电机控制报文返回

上一章中的力位混合模式、位置模式、速度模式、电流模式与 VF 模式发送控制报文后，均会返回状态报文，该返回报文中参数种类可选，共有 5 种返回报文，除默认返回报文外，其余四种均为测试帧方便检查数据使用。

5.1 默认运动模式状态返回报文

运动模式问答（接收） CANID: 关节 ID dlc: 16			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x80	当前电机模式+错误码 (uint4+uint12)		当前位置 (float)31~24 位
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
当前位置 (float)23~16 位	当前位置 (float)15~8 位	当前位置 (float)7~0 位	当前速度 (float)31~24 位
Byte 9	***Byte 10***	***Byte 11***	***Byte 12***
当前速度 (float)23~16 位	当前速度 (float)15~8 位	当前速度 (float)7~0 位	当前电流 (int16)15~8 位
Byte 13	***Byte 14***	***Byte 15***	***Byte 16***

当前电流 (int16)7~0 位	电机温度 (uint8)7~0 位	MOS 温度 (uint8)7~0 位	母线电压 (uint8)7~0 位

电机运动模式下，默认返回的报文格式。第二、三字节为电机模式与错误码拼接，高四位表示电机模式，即在二进制下的 0000 表示失能、0001 表示使能、0010 表示力位混合模式、0011 表示位置模式、0100 表示速度模式、0101 表示电流模式，其余保留；后 12 位按位或得到当前错误码，即由 12 种错误互相或得到的 uint12，当二进制下的 0000 0000 0000 表示正常运行，从高位到低位若有 1 则分别表示：bit11-指令超限、bit10-母线电压过压、bit9-母线电压欠压、bit8-控制板温度过温、bit7-电机绕组温度过温、bit6-电机堵转、bit5-电机绕组过流、bit4-编码器数据错误、bit3~0-保留；当前位置、当前速度为 float 型；当前电流为 int16 型，需要对接收数据除以 100 以获得真实电流值，精度 0.01A；电机温度、MOS 温度为 uint8 型，需要对接收数据减 50 以获得真实温度值，精度 1°C；母线电压为 uint8 型，需要对接收数据除以 2 以获得真实电压值，精度 0.5V。

5.2 切换返回报文类型

测试帧切换（发送） CANID: 关节 ID dlc: 2			
Byte 1	***Byte 2***		
0x20	新测试帧		

测试帧切换（接收） CANID: 关节 ID dlc: 2			
Byte 1	***Byte 2***		
0xA0	新测试帧		

切换测试帧返回，0 为默认返回报文，1~4 为下文的测试帧返回格式。

5.3 测试帧 1 返回

测试帧 1（接收）

CANID：关节 ID　dlc: 16			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0xA1	目标位置(int16) 15~8 位	目标位置(int16) 7~0 位	当前位置(int16) 15~8 位
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
当前位置(int16) 7~0 位	目标速度 (int16)15~8 位	目标速度 (int16)7~0 位	当前速度(int16) 15~8 位
Byte 9	***Byte 10***	***Byte 11***	***Byte 12***
当前速度(int16) 7~0 位	目标电流 (int16)15~8 位	目标电流 (int16)7~0 位	当前电流 (int16)15~8 位
Byte 13	***Byte 14***	***Byte 15***	***Byte 16***
当前电流 (int16)7~0 位	力矩系数 (uint16) 15~8 位	力矩系数 (uint16)7~0 位	前馈力矩 (int8)7~0 位

目标位置、当前位置、目标速度、当前速度、目标电流、实际电流为 int16 型，需要对接收数据除以 100 以获得真实值；力矩系数为 uint16 型，需要对接收数据除以 100 以获得真实值；前馈力矩为 int8 型，无需数值计算。

5.4 测试帧 2 返回

测试帧 2（接收） CANID：关节 ID　dlc: 16			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0xA2	目标电流 (int16)15~8 位	目标电流 (int16)7~0 位	当前电流 (float)31~24 位
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
当前电流 (float)23~16 位	当前电流 (float)15~8 位	当前电流 (float)7~0 位	电机温度 (float)31~24 位
Byte 9	***Byte 10***	***Byte 11***	***Byte 12***

电机温度 (float)23~16 位	电机温度 (float)15~8 位	电机温度 (float)7~0 位	MOS 温度 (float)31~24 位
Byte 13	***Byte 14***	***Byte 15***	***Byte 16***
MOS 温度 (float)23~16 位	MOS 温度 (float)15~8 位	MOS 温度 (float)7~0 位	自增计数

目标电流为 int16 型，需要对接收数据除以 100 以获得真实值；当前电流、电机温度、MOS 温度为 float 型；自增计数为发送的控制指令中自增计数原样返回，以供上位机判断是否丢包。

5.4 测试帧 3 返回

测试帧 3（接收） CANID: 关节 ID dlc: 13			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0xA3	关节零点位置 (float)31~24 位	关节零点位置 (float)23~16 位	关节零点位置 (float) 15~8 位
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
关节零点位置 float) 7~0 位	电机绝对位置 (float)31~24 位	电机绝对位置 (float)23~16 位	电机绝对位置 (float) 15~8 位
Byte 9	***Byte 10***	***Byte 11***	***Byte 12***
电机绝对位置 (float) 7~0 位	减速器绝对位置 (float)31~24 位	减速器绝对位置 (float)23~16 位	减速器绝对位置 (float) 15~8 位
Byte 13			
减速器绝对位置 (float) 7~0 位			

关节零点位置、电机绝对位置、减速器绝对位置为 float 型。

5.5 测试帧 4 返回

测试帧 4（接收） CANID: 关节 ID dlc: 4			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0xA4	减速比 (uint8)7~0 位	I_Gain (uint16) 15~8 位	I_Gain (uint16) 7~0 位

关节零点位置、电机绝对位置、减速器绝对位置为 float 型。

5.6 测试帧 5 返回（部分固件可能未更新该功能）

测试帧 5（接收） CANID: 关节 ID dlc: 13			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0xA5	上次报文秒数 (uint32)31~24 位	上次报文秒数 (uint32)23~16 位	上次报文秒数 (uint32)15~8 位
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
上次报文秒数 (uint32)7~0 位	上次报文毫秒数 (uint16)15~8 位	上次报文毫秒数 (uint16)7~0 位	本次报文秒数 (uint32)31~24 位
Byte 9	***Byte 10***	***Byte 11***	***Byte 12***
本次报文秒数 (uint32)23~16 位	本次报文秒数 (uint32)15~8 位	本次报文秒数 (uint32)7~0 位	本次报文毫秒数 (uint16)15~8 位
Byte 13			
本次报文毫秒数			

(uint16)7~0 位			

测试帧 5 返回当前 MCU 总运行时间（总运行时间体现为秒数加毫秒数）及上次收到 CAN 报文（仅对本关节 ID 的报文记录）时间，以方便使用者检测发送频率及通过总运行时间判断是否有断电重启现象。

5.7 自动报文（部分固件可能未更新该功能）

自动报文设置（发送） CANID：关节 ID　dlc: 3			
Byte 1	***Byte 2***	***Byte 3***	
0x40	自动报文频率（uint16）15~8 位	自动报文频率（uint16）7~0 位	

该指令设置自动报文频率，该频率默认为 0，为 0 时关闭自动报文功能，大于 0 时开启自动报文功能，当自动报文功能开启时，发送第四章的控制指令不会返回状态报文。

除了关节上电默认自动报文关闭外，若关节 100ms 未收到 CAN 指令而进入停机模式，或收到停机指令，也会关闭自动报文功能，可再通过开启自动报文并设置频率恢复。

自动报文设置（接收） CANID：关节 ID　dlc: 3			
Byte 1	***Byte 2***	***Byte 3***	
0xC0	自动报文频率（uint16）15~8 位	自动报文频率（uint16）7~0 位	

6.Bootloader 升级说明

原则上，关节装壳后仅允许使用上位机进行通过 CAN 的关节固件升级，但允许在二次开发中进行 Bootloader 功能中的关节版本号查询，以下是 Bootloader CAN 升级报文说明，具体出现了哪些功能视该文档面向对象而定。

6.1 查询关节版本号

每款关节均有其版本号可供查询，版本号由关节程序及升级文件定义，关节版本号长度为 32 位，以每 8 位表示其中一种含义，从高至低分别表示：关节类型、机械版本、硬件版本、软件版本。关节类型用来表示关节外径尺寸(mm)；机械版本代表减速器类型，0x00 代表 H 系列、0x10 代表 P 系列、0x20 代表 C 系列；硬件版本和软件版本代表迭代版本，当使用 Bootloader CAN 升级时，只允许对前三个字节完全相同的固件进行升级，如对 55H 关节使用 55P 的固件，则会在升级中出现错误，升级中断并报错。

对指定 ID 查版本号（发送） CANID：关节 ID+0x400　dlc: 1			
Byte 1			
0x01			

对指定 ID 查版本号（发送） CANID：关节 ID+0x400　dlc: 5			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x81	版本号 31~24 位	版本号 23~16 位	版本号 15~8 位
Byte 5			
版本号 7~0 位			

6.2 新版本发送与核对

此为 Bootloader 升级流程第一步。通过 CAN 报文，将新固件的版本号向关节发送，关

节端进行版本号核对，若符合同关节版本号规格则原样返回，不符合则返回错误报文。接收到正确返回报文后可发送升级信息报文。

新版本信息（发送） CANID：关节 ID+0x400　dlc: 5			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x02	新版本号 31~24 位	新版本号 23~16 位	新版本号 15~8 位
Byte 5			
新版本号 7~0 位			

核对关节成功（发送） CANID：关节 ID+0x400　dlc: 5			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x82	新版本号 31~24 位	新版本号 23~16 位	新版本号 15~8 位
Byte 5			
新版本号 7~0 位			

核对关节失败（发送） CANID：关节 ID+0x400　dlc: 2			
Byte 1	***Byte 2***		
0x80	0x80		

6.3 升级信息

此为 Bootloader 升级流程第二步。通过 CAN 报文发送升级数据的校验码及升级数据长度，作为升级数据的校验信息，防止在后续传输过程中出现丢包及数据损坏的可能。校验码是对升级数据的 CRC16 校验，其校验程序会在下文中给出，数据长度经计算最多使用 24 位表达。关节返回报文中，校验码和数据长度原样返回，后面增加一个对 Flash 数据保存区数

据的擦除进度百分比，需要连续对关节发送升级信息报文，当接收到的擦除进度为 0x64，即代表保存区已清空，可以用来存放新的数据。接收到完全擦除返回报文后可发送升级数据报文。

升级信息（发送） CANID：关节 ID+0x400　dlc: 6			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x03	CRC15~8 位	CRC7~0 位	Size23~16 位
Byte 5	***Byte 6***		
Size15~8 位	Size7~0 位		

升级信息（发送） CANID：关节 ID+0x400　dlc: 7			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x83	CRC15~8 位	CRC7~0 位	Size23~16 位
Byte 5	***Byte 6***	***Byte 7***	
Size15~8 位	Size7~0 位	擦除进度	

使用固件数据（uint8）和固件长度（uint32）计算固件 CRC

CRCcalculated = crc16Forcan(datareceive, newdata.datasize);

CRC 表格

```
static const uint16_t crc16Table[] =
{
    0x0000, 0xC0C1, 0xC181, 0x0140, 0xC301, 0x03C0, 0x0280, 0xC241,
    0xC601, 0x06C0, 0x0780, 0xC741, 0x0500, 0xC5C1, 0xC481, 0x0440,
    0xCC01, 0x0CC0, 0x0D80, 0xCD41, 0x0F00, 0xCFC1, 0xCE81, 0x0E40,
    0x0A00, 0xCAC1, 0xCB81, 0x0B40, 0xC901, 0x09C0, 0x0880, 0xC841,
    0xD801, 0x18C0, 0x1980, 0xD941, 0x1B00, 0xDBC1, 0xDA81, 0x1A40,
    0x1E00, 0xDEC1, 0xDF81, 0x1F40, 0xDD01, 0x1DC0, 0x1C80, 0xDC41,
```

0x1400, 0xD4C1, 0xD581, 0x1540, 0xD701, 0x17C0, 0x1680, 0xD641,
0xD201, 0x12C0, 0x1380, 0xD341, 0x1100, 0xD1C1, 0xD081, 0x1040,
0xF001, 0x30C0, 0x3180, 0xF141, 0x3300, 0xF3C1, 0xF281, 0x3240,
0x3600, 0xF6C1, 0xF781, 0x3740, 0xF501, 0x35C0, 0x3480, 0xF441,
0x3C00, 0xFCC1, 0xFD81, 0x3D40, 0xFF01, 0x3FC0, 0x3E80, 0xFE41,
0xFA01, 0x3AC0, 0x3B80, 0xFB41, 0x3900, 0xF9C1, 0xF881, 0x3840,
0x2800, 0xE8C1, 0xE981, 0x2940, 0xEB01, 0x2BC0, 0x2A80, 0xEA41,
0xEE01, 0x2EC0, 0x2F80, 0xEF41, 0x2D00, 0xEDC1, 0xEC81, 0x2C40,
0xE401, 0x24C0, 0x2580, 0xE541, 0x2700, 0xE7C1, 0xE681, 0x2640,
0x2200, 0xE2C1, 0xE381, 0x2340, 0xE101, 0x21C0, 0x2080, 0xE041,
0xA001, 0x60C0, 0x6180, 0xA141, 0x6300, 0xA3C1, 0xA281, 0x6240,
0x6600, 0xA6C1, 0xA781, 0x6740, 0xA501, 0x65C0, 0x6480, 0xA441,
0x6C00, 0xACC1, 0xAD81, 0x6D40, 0xAF01, 0x6FC0, 0x6E80, 0xAE41,
0xAA01, 0x6AC0, 0x6B80, 0xAB41, 0x6900, 0xA9C1, 0xA881, 0x6840,
0x7800, 0xB8C1, 0xB981, 0x7940, 0xBB01, 0x7BC0, 0x7A80, 0xBA41,
0xBE01, 0x7EC0, 0x7F80, 0xBF41, 0x7D00, 0xBDC1, 0xBC81, 0x7C40,
0xB401, 0x74C0, 0x7580, 0xB541, 0x7700, 0xB7C1, 0xB681, 0x7640,
0x7200, 0xB2C1, 0xB381, 0x7340, 0xB101, 0x71C0, 0x7080, 0xB041,
0x5000, 0x90C1, 0x9181, 0x5140, 0x9301, 0x53C0, 0x5280, 0x9241,
0x9601, 0x56C0, 0x5780, 0x9741, 0x5500, 0x95C1, 0x9481, 0x5440,
0x9C01, 0x5CC0, 0x5D80, 0x9D41, 0x5F00, 0x9FC1, 0x9E81, 0x5E40,
0x5A00, 0x9AC1, 0x9B81, 0x5B40, 0x9901, 0x59C0, 0x5880, 0x9841,
0x8801, 0x48C0, 0x4980, 0x8941, 0x4B00, 0x8BC1, 0x8A81, 0x4A40,
0x4E00, 0x8EC1, 0x8F81, 0x4F40, 0x8D01, 0x4DC0, 0x4C80, 0x8C41,
0x4400, 0x84C1, 0x8581, 0x4540, 0x8701, 0x47C0, 0x4680, 0x8641,
0x8201, 0x42C0, 0x4380, 0x8341, 0x4100, 0x81C1, 0x8081, 0x4040
};

CRC 计算程序，使用查表法

uint16_t crc16forcan(uint8_t *buff, uint32_t bufflen)

```
{
    uint8_t buf;
    uint16_t crc16 = 0xFFFF;
    int i;

    for (i = 0; i < bufflen; i++)
    {
        buf = buff[i] ^ crc16;
        crc16 >>= 8;
        crc16 ^= crc16Table[buf];
    }

    return  crc16;
}
```

6.4 升级数据

此为 Bootloader 升级流程第三步。使用自增计数向关节发送升级数据，每包报文发送 12 字节的升级数据，以 bin 文件按顺序发送（不用切换大小端）。自增计数从 0x00 0x00 开始，每次加 1，即第二包为 0x00 0x01，若超过 FFFF 则加一溢出为 0000。关节会原样将接收报文返回，所有数据传输完成后可发送数据下发完成报文。

升级数据（发送） CANID：关节 ID+0x400　dlc: 15			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x04	自增计数 15~8	自增计数 7~0 位	Data0

	位		
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
Data1	Data2	Data3	Data4
Byte 9	***Byte 10***	***Byte 11***	***Byte 12***
Data5	Data6	Data7	Data8
Byte 13	***Byte 14***	***Byte 15***	
Data9	Data10	Data11	

升级数据（发送） CANID：关节 ID+0x400　dlc: 15			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x84	自增计数 15~8 位	自增计数 7~0 位	Data0
Byte 5	***Byte 6***	***Byte 7***	***Byte 8***
Data1	Data2	Data3	Data4
Byte 9	***Byte 10***	***Byte 11***	***Byte 12***
Data5	Data6	Data7	Data8
Byte 13	***Byte 14***	***Byte 15***	
Data9	Data10	Data11	

6.5 数据下发完成

此为 Bootloader 升级流程第四步。当数据全部传输完毕后，关节端收到数据下发完成报文便会根据之前发送的数据长度和接收数据进行校验码计数，若其与之前接受的校验码匹配，则表明未发生数据丢失或错误，将接收数据存至已清空的 Flash 保存区，同时改变版本号为接收到的版本号。Flash 操作完成后，会自动重启进行固件程序的更新。关节返回报文后，led 灯会有一定延迟时间不亮，等到重新点亮并显示正常运行后，可发送查询重启情况指令。

数据下发完成（发送） CANID：关节 ID+0x400　dlc: 1			
Byte 1			
0x05			

数据下发完成（发送） CANID：关节 ID+0x400　dlc: 7			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x85	CRC15~8 位	CRC7~0 位	新版本号 31~24 位
Byte 5	***Byte 6***	***Byte 7***	
新版本号 23~16 位	新版本号 15~8 位	新版本号 7~0 位	

6.6 查询重启情况

此为 Bootloader 升级流程第五步。关节重启后会对升级标志位做修改，对关节发送查询重启以确定关节程序为新传输的数据，发送返回报文中包含程序版本号，之后对升级标志位擦除。可发送升级完成报文。

查询重启情况（发送） CANID：关节 ID+0x400　dlc: 1			
Byte 1			
0x06			

查询重启情况（发送） CANID：关节 ID+0x400　dlc: 5			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***

0x86	新版本号 31~24 位	新版本号 23~16 位	新版本号 15~8 位
Byte 5			
新版本号 7~0 位			

6.7 升级完成

此为 Bootloader 升级流程第六步。关节接收报文后，检查升级标志位是否清除以确保可再次升级。升级标志 0x00 为升级标志位未擦除，0x01 为已擦除，接收 0x01 代表升级流程成功结束。

升级完成（发送） CANID：关节 ID+0x400　dlc: 1			
Byte 1			
0x07			

升级完成（发送） CANID：关节 ID+0x400　dlc: 6			
Byte 1	***Byte 2***	***Byte 3***	***Byte 4***
0x87	新版本号 31~24 位	新版本号 23~16 位	新版本号 15~8 位
Byte 5	***Byte 6***		
新版本号 7~0 位	升级标志		

更新记录：

V1.0-初版

V1.1-增加文字说明解释协议升级需求

V1.2-修改查询 ID/重置 ID 报文内容，使其减少相互识别风险。修改发送 MCUID 使其全部发送。

V1.3-20251029-修改返回报文时电机模式及错误码说明

20251208-互换错误码的欠压过压

V1.4-20260120-增加测试帧 5 及自动报文

V1.5-20260203-增加固件升级的 CRC 表格、程序和说明

V1.6-20260331-修改力位混合模式 KPKD 值缩放倍数，现在为 10 倍。